Ohjelmistotuotanto

Syksy 2026

Luento 5

9.11.2026

Kurssipalaute

- Kurssipalaute
  - Kurssilla lopussa kerättävän palautteen lisäksi ns. jatkuva palaute https://norppa.helsinki.fi

Paja

- BK107
  - ma 14.30-16.30
  - ti 12-16
  - to 12-16
  - pe 12-14

Miniprojektit

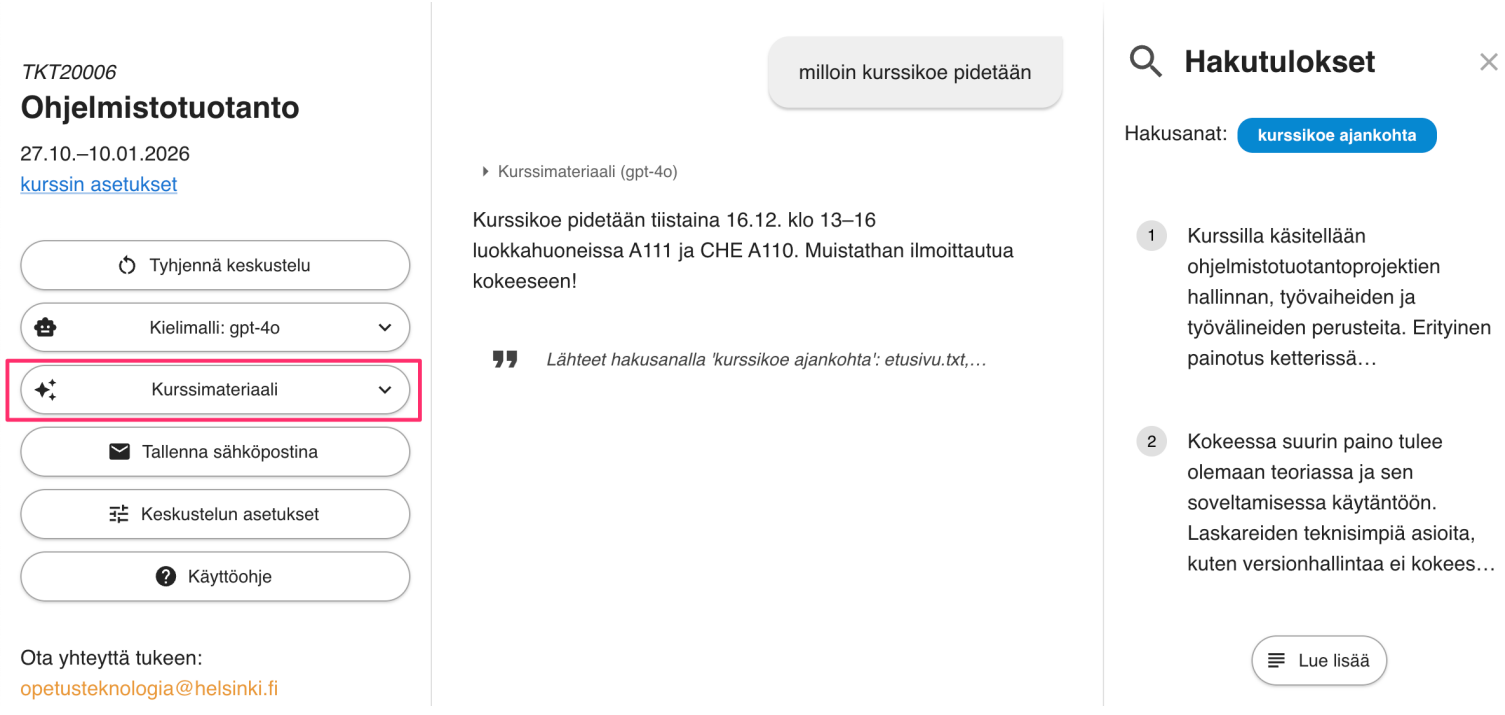
- Käynnistyvät **tänään!**
- Aloitustilaisuudet
  - ma 14-16
  - ti 14-16
  - ke 10-12
  - ke 12-14
  - to 14-16
  - to 16-18
- Seuraavat viikot: sprinttien katselmus ja suunnittelu samassa aikaikkunassa
- Loppudemot
  - **ke 9.12. klo 10-12 B123**
  - **to 10.12. klo 10-12 A111**

CurreChat

- Käytettävissä HY:n GPT-chat
  - https://curre.helsinki.fi/chat

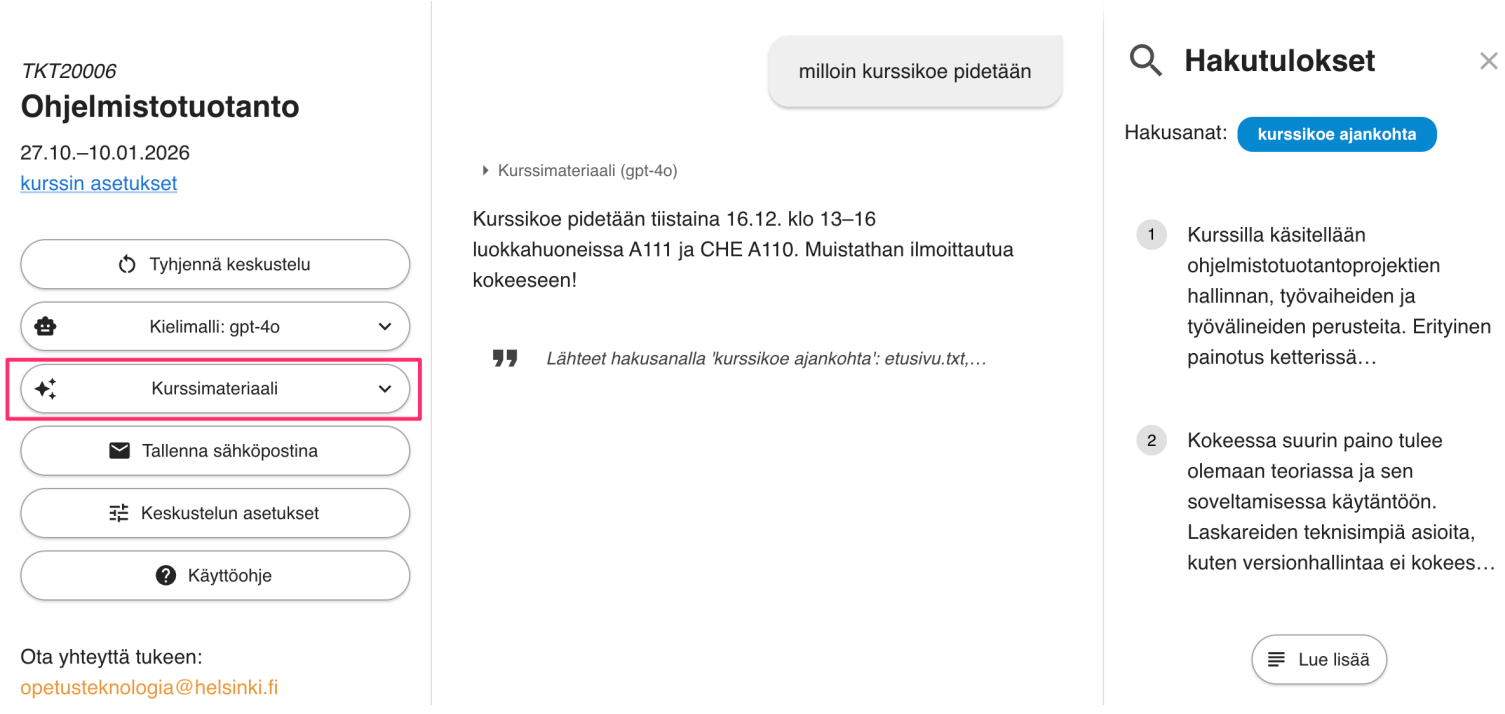
CurreChat

- Käytettävissä HY:n GPT-chat
  - https://curre.helsinki.fi/chat



CurreChat

- Käytettävissä HY:n GPT-chat
  - https://curre.helsinki.fi/chat



- Ei (ehkä) hallusinoi
- Osin herkkä kysymysten sanamuodon suhteen

Ohjelmiston elinkaari (software lifecycle)

- Vaatimusten analysointi ja määrittely
- Suunnittelu
- Toteutus
- **Testaus ja laadunhallinta**
- Ohjelmiston ylläpito ja evoluutio

Ohjelmistojen laadunhallinta

- Laadunhallinnan kaksi oleellista kysymystä
  - verifiointi: *are we building the product right*
  - validointi: *are we building the right product*

Ohjelmistojen laadunhallinta

- Laadunhallinnan kaksi oleellista kysymystä
  - verifiointi: *are we building the product right*
  - validointi: *are we building the right product*
- **Verifiointi**: toteuttaako ohjelmisto vaatimusmäärittelyn aikana asetetut vaatimukset
  - Testataan toiminnalliset ja ei-toiminnalliset vaatimukset

Ohjelmistojen laadunhallinta

- Laadunhallinnan kaksi oleellista kysymystä
  - verifiointi: *are we building the product right*
  - validointi: *are we building the right product*
- **Verifiointi**: toteuttaako ohjelmisto vaatimusmäärittelyn aikana asetetut vaatimukset
  - Testataan toiminnalliset ja ei-toiminnalliset vaatimukset
- **Validointi**: täyttääkö ohjelmisto käyttäjän odotukset
  - Vaatimusmäärittelyn ei aina ole kirjattu sitä mitä käyttäjä todella tarvitsee

Verifiointi ja Validointi

- Varmistetaan että ohjelma on *riittävän hyvä* käyttötarkoitukseensa
  - Hyvyys on suhteellista ja riippuu ohjelman käyttötarkoituksesta
  - Ohjelman ei yleensä tarvitse olla virheetön ollakseen riittävän hyvä käytettäväksi

Verifiointi ja Validointi

- Varmistetaan että ohjelma on *riittävän hyvä* käyttötarkoitukseensa
  - Hyvyys on suhteellista ja riippuu ohjelman käyttötarkoituksesta
  - Ohjelman ei yleensä tarvitse olla virheetön ollakseen riittävän hyvä käytettäväksi
- Verifioinnin ja validoinnin suorittamista käytetään yleisesti nimitystä *laadunhallinta* (engl. quality assurance, QA)
  - Jos laadunhallinta on erillisen tiimin vastuulla, käytetään tästä usein nimitystä *QA-tiimi*

Verifioinnin ja Validoinnin tekniikat

- Verifioinnissa käytetään kahta erilaista tekniikkaa
  - Katselmointia
  - Testausta

Verifioinnin ja Validoinnin tekniikat

- Verifioinnissa käytetään kahta erilaista tekniikkaa
  - Katselmointia
  - Testausta
- **Katselmoinneissa** (review) käydään läpi tuotantoprosessin aikana tehtyjä dokumentteja ja ohjelmakoodia, ja etsitään näistä ongelmia

Verifioinnin ja Validoinnin tekniikat

- Verifioinnissa käytetään kahta erilaista tekniikkaa
  - Katselmointia
  - Testausta
- **Katselmoinneissa** (review) käydään läpi tuotantoprosessin aikana tehtyjä dokumentteja ja ohjelmakoodia, ja etsitään näistä ongelmia
- *Staattinen tekniikka*, suorituskelpoista koodia ei tarvita
  - Jos kohteena on ohjelmakoodi, ei sitä katselmoinnissa suoriteta

Verifioinnin ja Validoinnin tekniikat

- Verifioinnissa käytetään kahta erilaista tekniikkaa
  - Katselmointia
  - Testausta
- **Katselmoinneissa** (review) käydään läpi tuotantoprosessin aikana tehtyjä dokumentteja ja ohjelmakoodia, ja etsitään näistä ongelmia
- *Staattinen tekniikka*, suorituskelpoista koodia ei tarvita
  - Jos kohteena on ohjelmakoodi, ei sitä katselmoinnissa suoriteta
- **Testausksessa** tarkkaillaan miten ohjelma reagoi annettuihin testisyötteisiin
  - *dynaaminen tekniikka*, edellyttää ohjelmakoodin suorittamista

Katselmointi

Vaatimusten validointi katselmoimalla dokumentaatiota

Vaatimusten validointi katselmoimalla dokumentaatiota

- Ohjelmistolle määritellyt vaatimukset on validoitava:
  - varmistettava, että määrittelydokumentti määrittelee oikeanlaisen ohjelmiston

Vaatimusten validointi katselmoimalla dokumentaatiota

- Ohjelmistolle määritellyt vaatimukset on validoitava:
  - varmistettava, että määrittelydokumentti määrittelee oikeanlaisen ohjelmiston
- Vesiputousmallissa määrittelydokumentin kirjattujen vaatimusten validointi suoritetaan *katselmoimalla*

Vaatimusten validointi katselmoimalla dokumentaatiota

- Ohjelmistolle määritellyt vaatimukset on validoitava:
  - varmistettava, että määrittelydokumentti määrittelee oikeanlaisen ohjelmiston
- Vesiputousmallissa määrittelydokumentin kirjattujen vaatimusten validointi suoritetaan *katselmoimalla*
- Vaatimusmäärittelyn lopuksi asiakas tarkastaa vastaako määrittelydokumentti mielikuvaa tilattavasta järjestelmästä

Vaatimusten validointi katselmoimalla dokumentaatiota

- Ohjelmistolle määritellyt vaatimukset on validoitava:
  - varmistettava, että määrittelydokumentti määrittelee oikeanlaisen ohjelmiston
- Vesiputousmallissa määrittelydokumentin kirjattujen vaatimusten validointi suoritetaan *katselmoimalla*
- Vaatimusmäärittelyn lopuksi asiakas tarkastaa vastaako määrittelydokumentti mielikuvaa tilattavasta järjestelmästä
- Katselmoinnin jälkeen määrittelydokumentti jäädytetään
  - muuttaminen vaatii yleensä monimutkaista prosessia

Vaatimusten ketterä validointi

- Ketterässä ohjelmistotuotannossa vaatimusten validointi tapahtuu sprinttien päättävien demonstraatioiden yhteydessä

Vaatimusten ketterä validointi

- Ketterässä ohjelmistotuotannossa vaatimusten validointi tapahtuu sprinttien päättävien demonstraatioiden yhteydessä
- Asiakkaalle näytetään ohjelman toimivaa versiota

Vaatimusten ketterä validointi

- Ketterässä ohjelmistotuotannossa vaatimusten validointi tapahtuu sprinttien päättävien demonstraatioiden yhteydessä
- Asiakkaalle näytetään ohjelman toimivaa versiota
- Asiakas voi itse verrata onko lopputulos haluttu
  - Jos ei, on seuraavassa sprintissä mahdollista ottaa korjausliike

Koodin katselmointi

Koodin katselmointi

- Koodin katselmointi eli lukeminen jonkun muun kuin ohjelmoijan toimesta on tutkitusti tehokas keino laadun parantamisessa

Koodin katselmointi

- Koodin katselmointi eli lukeminen jonkun muun kuin ohjelmoijan toimesta on tutkitusti tehokas keino laadun parantamisessa
- Voidaan havaita koodista ongelmia, joita testaus ei havaitse
  - noudattaako koodi sovittua tyyliä
  - onko koodi ylläpidettävää
  - onko koodissa tietoturvan kannalta vaarallisia piirteitä

Koodin katselmointi

- Koodin katselmointi eli lukeminen jonkun muun kuin ohjelmoijan toimesta on tutkitusti tehokas keino laadun parantamisessa
- Voidaan havaita koodista ongelmia, joita testaus ei havaitse
  - noudattaako koodi sovittua tyyliä
  - onko koodi ylläpidettävää
  - onko koodissa tietoturvan kannalta vaarallisia piirteitä
- Perinteisesti käyty läpi onko koodissa tiettyjä checklisteissä listattuja riskialttiita piirteitä

V1	Variables are lower case words separated by underscores (`this_is_a_var`)
V2	Constants are upper case words separated by underscores. (`THIS_IS_A_CONST`)
V3	No **int** declarations. (Use **uint8, int8, uint16, int16, uint32, int32** instead.)
V4	One declaration per line. (Exception: Highly coupled variables, i.e. `width` and `height`.)
V5	All declarations have a comment after them explaining the variable.
V6	Units are declared when appropriate.
V7	No hidden variables. That is, a variable defined in an inner block may not have the same name as variable in an outer block.
V8	Never use "O" (Capital O) or "l" (lower case "l") for variable or constant names.

Staattinen analyysi

- Nykyään on tarjolla katselmointia automatisoivia *staattista analyysiä* tekeviä työkaluja
  - Pythonilla Pylint ja yhä suositumpi Ruff
  - JavaScriptilla ESlint

Staattinen analyysi

- Nykyään on tarjolla katselmointia automatisoivia *staattista analyysiä* tekeviä työkaluja
  - Pythonilla Pylint ja yhä suositumpi Ruff
  - JavaScriptilla ESlint
- Myös pilvipalveluna toimivia työkaluja (esim. Qlty)
  - Suorittavat tarkastukset aina kun uutta koodia pushataan GitHubiin
  - Huomaavat koodin laadun muutoksista, esim. jos koodin kompleksisuus kasvaa muutosten yhteydessä

Staattinen analyysi

- Nykyään on tarjolla katselmointia automatisoivia *staattista analyysiä* tekeviä työkaluja
  - Pythonilla Pylint ja yhä suositumpi Ruff
  - JavaScriptilla ESlint
- Myös pilvipalveluna toimivia työkaluja (esim. Qlty)
  - Suorittavat tarkastukset aina kun uutta koodia pushataan GitHubiin
  - Huomaavat koodin laadun muutoksista, esim. jos koodin kompleksisuus kasvaa muutosten yhteydessä
- GitHubiin sisäänrakennettuna
  - CodeQL etsii tietoturva-aukkoja
  - Dependabot varoittaa haavoittuvista riippuvuuksista
  - salaisuuksien skannaus varoittaa repositorioon päätyneistä API-avaimista ja salasanoista

Esimerkki qlty.sh

oodikone master

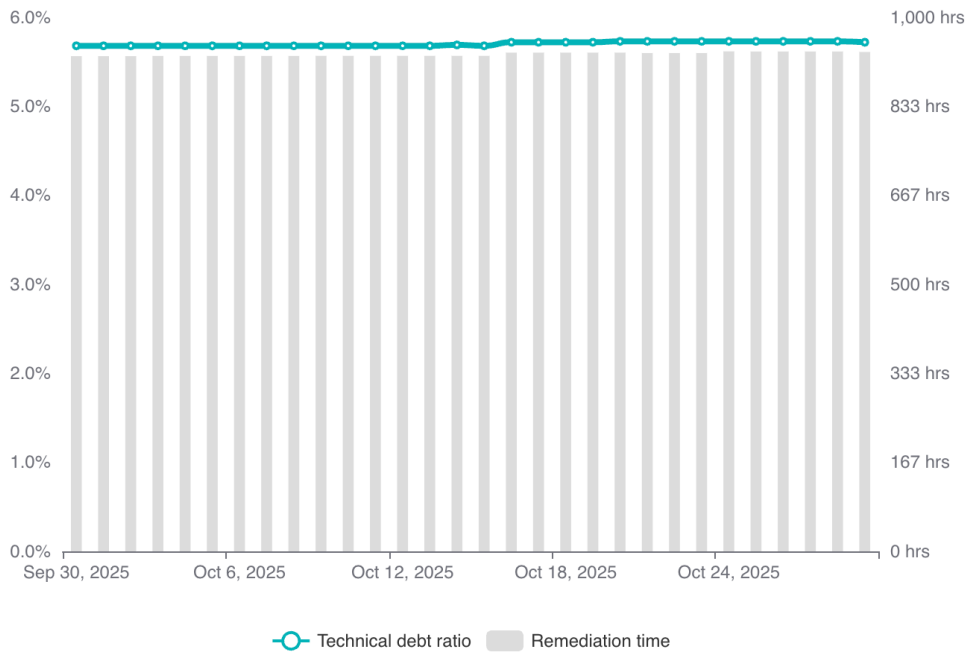
TypeScript | 58 KLOC | Star

Overview | Issues | Hotspots NEW | Components NEW | Metrics | Trends | Pull Requests | Settings | Last build 1 day ago

B Maintainability
272 code smells

A Coverage
0.0% covered lines

A Security
No open issues





Technical debt ratio
5.72%

Remediation time
4 mos

Duplication
1.2%

Complexity / KLOC
139

Maintainability hotspots

studyTrackStats.ts	F
facultyGraduationTimes.ts	F
index.tsx	F
index.tsx	F
facultyBasics.ts	F

TYPE	FILE ^	SIZE	LOC	ISSUES	MAINTAINABILITY	COMPLEXITY	DUPLICATION
	..						
	config						
	database	S	1,928	3	A 4 hrs	32	2.7%
	environment						
	middleware	XS	190	1	A 35 mins	39	0.0%
	models	S	1,020	5	A 1 hr	1	6.1%
	rapodiff	XS	198	2	A 1 hr	38	0.0%
	routes	S	2,205	7	C 4 days	523	2.3%
	services	S	8,862	61	D 2 mos	1,892	1.8%
	types	XS	42	0	A 0 mins	0	0.0%
	util	XS	325	0	A 0 mins	80	0.0%
	worker	XS	143	0	A 0 mins	7	0.0%

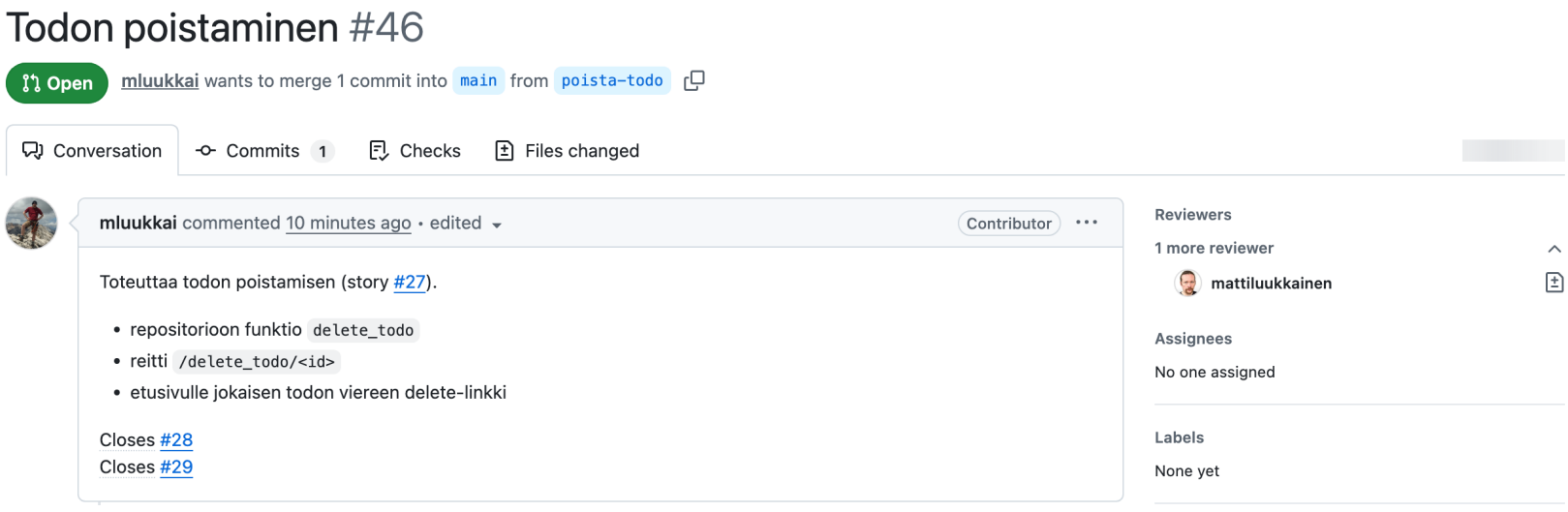
Koodin katselmointi: GitHub ja pull requestit

- GitHubin *pull requestit* tarjoavat hyvän työkalun koodikatselmointiin

Koodin katselmointi: GitHub ja pull requestit

- GitHubin *pull requestit* tarjoavat hyvän työkalun koodikatselmointiin
- Työn kulku on seuraava
  - Sovelluskehittäjä tekee muutokset omaan haaraansa (branch) ja avaa siitä pull requestin
  - open source -projekteissa ulkopuolinen kehittäjä tekee muutokset forkkaamaansa kopioon
  - Joku, esim. *senior developer* tekee katselmoinnin pull requestille
  - Jos koodi ei ole riittävän hyvää, annetaan pull requestin tekijälle parannusehdotuksia
  - Muutosten ollessa hyväksyttäviä pull request yhdistetään päärepositorioon

Esimerkki: pull request miniprojektin boilerplateen



- *Closes #28*: mergeäminen sulkee storyn taskin

Katselmoija pyytää korjauksia



mattiluukkainen suggested changes now

View reviewed changes

mattiluukkainen left a comment

Hyvä alku! Poisto pitää tehdä POST-pyynnöllä, ks. kommentit. Definition of donen mukaan storylle tarvitaan myös Robot-testit (#30). Lisäätkö ne tähän PR:ään?

src/templates/index.html

```
18  18          <button type="submit">mark as done</button>
19  19        </form>
20  20        {% endif %}
    21  +     <a href="/delete_todo/{{ todo.id }}" style="margin-left: 10px">delete</a>
```



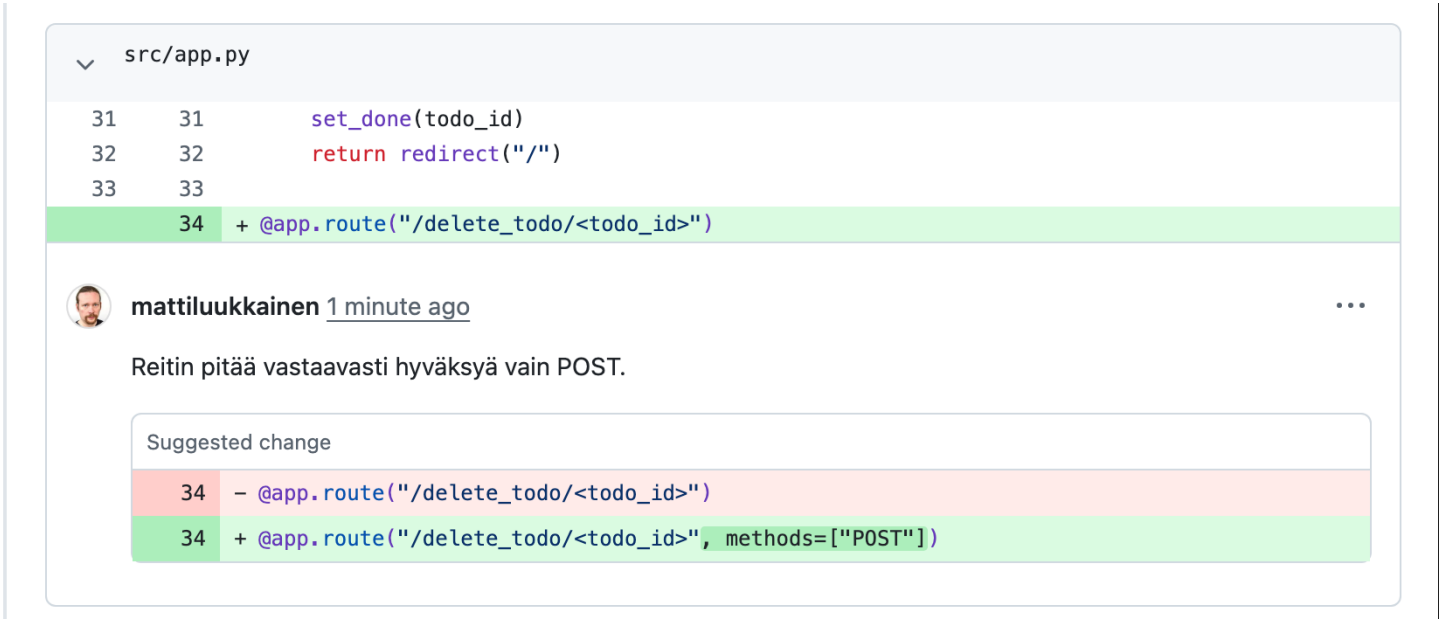
mattiluukkainen 3 minutes ago

Poisto GET-pyynnöllä on riskialtis: esim. selaimen linkkien esilataus voi poistaa todoja vahingossa. Tehdään poisto lomakkeella POST-pyyntönä samaan tapaan kuin mark as done.

Suggested change

```
21  -     <a href="/delete_todo/{{ todo.id }}" style="margin-left: 10px">delete</a>
21  +     <form action="/delete_todo/{{ todo.id }}" method="post" style="margin-left: 10px">
22  +       <button type="submit">delete</button>
23  +     </form>
```

Ehdotettu muutos (suggested change)



src/app.py

```
31  31      set_done(todo_id)
32  32      return redirect("/")
33  33
    34  + @app.route("/delete_todo/<todo_id>")
```

mattiluukkainen 1 minute ago

Reitin pitää vastaavasti hyväksyä vain POST.

Suggested change

```
34  - @app.route("/delete_todo/<todo_id>")
34  + @app.route("/delete_todo/<todo_id>", methods=["POST"])
```

- Tekijä voi hyväksyä ehdotuksen yhdellä napin painalluksella

Katselmoinnin hyödyt

- Microsoftilla tehty tutkimus (Bacchelli & Bird 2013)
  - kehittäjät pitivät virheiden löytämistä katselmoinnin päätavoitteena
  - valtaosa kommenteista koski kuitenkin ylläpidettävyyttä ja ymmärrettävyyttä

Katselmoinnin hyödyt

- Microsoftilla tehty tutkimus (Bacchelli & Bird 2013)
  - kehittäjät pitivät virheiden löytämistä katselmoinnin päätavoitteena
  - valtaosa kommenteista koski kuitenkin ylläpidettävyyttä ja ymmärrettävyyttä
- Tärkeitä hyötyjä myös tiedon levittäminen tiimissä ja vaihtoehtoisten ratkaisujen löytäminen
  - samansuuntaiset tulokset Googlella (Sadowski ym. 2018)

Katselmoinnin hyödyt

- Microsoftilla tehty tutkimus (Bacchelli & Bird 2013)
  - kehittäjät pitivät virheiden löytämistä katselmoinnin päätavoitteena
  - valtaosa kommenteista koski kuitenkin ylläpidettävyyttä ja ymmärrettävyyttä
- Tärkeitä hyötyjä myös tiedon levittäminen tiimissä ja vaihtoehtoisten ratkaisujen löytäminen
  - samansuuntaiset tulokset Googlella (Sadowski ym. 2018)
- Tärkein yksittäinen asia: **pidä pull requestit pieninä**
  - pienen muutoksen voi katselmoida huolellisesti
  - tuhannen rivin muutokselle annetaan helposti pelkkä “LGTM”

Katselmointi on vuorovaikutustilanne

- Virheiden osoittaminen ja eriävä mielipide ovat ihmissuhteisiin liittyviä riskejä
  - niin katselmoijalle kuin koodin kirjoittajalle

Katselmointi on vuorovaikutustilanne

- Virheiden osoittaminen ja eriävä mielipide ovat ihmissuhteisiin liittyviä riskejä
  - niin katselmoijalle kuin koodin kirjoittajalle
- Kommentit kannattaa kohdistaa koodiin, ei sen kirjoittajaan

Katselmointi on vuorovaikutustilanne

- Virheiden osoittaminen ja eriävä mielipide ovat ihmissuhteisiin liittyviä riskejä
  - niin katselmoijalle kuin koodin kirjoittajalle
- Kommentit kannattaa kohdistaa koodiin, ei sen kirjoittajaan
- Katselmoinnin voi tehdä myös yhdessä
  - molemmat voivat perustella ratkaisujaan ja kasvattaa ymmärrystään

GitHub copilotin tekemä koodin katselmointi

GitHub copilotin tekemä koodin katselmointi

- Pull requestin tekemien muutosten yleiskatsaus

Copilot AI left a comment

Pull Request Overview

This pull request adds a UI management system for warehouse (Varasto) operations and integrates pre-commit hooks for code quality. The main changes include:

- New interactive CLI interface for managing multiple warehouses
- Integration of pre-commit with pylint for automated code quality checks
- Minor indentation fix in the core warehouse module

Reviewed Changes

Copilot reviewed 3 out of 5 changed files in this pull request and generated 3 comments.

▼ Show a summary per file

File	Description
src/varasto.py	Fixed indentation on line 29 in the `lisaa_varastoon` method
src/ui.py	Added new VarastoUI class providing a complete CLI interface for warehouse management operations
pyproject.toml	Added pre-commit as a project dependency

Copilot antaa myös detaljoidumpia kommentteja sekä parannusehdotuksia

src/ui.py

Comment on lines +137 to +138

```
137 +         print(f"Täyttöaste: {(varasto.saldo / varasto.tilavu
138 +
```

Copilot AI 13 minutes ago

Division by zero error will occur if `varasto.tilavuus` is 0. Although the Varasto class sets tilavuus to 0 when negative values are provided, this would cause a runtime error when displaying warehouse details for such cases.

Suggested change

```
137 -         print(f"Täyttöaste: {(varasto.saldo / varasto.tilavuus * 100):.1f}%")
137 +         if varasto.tilavuus == 0:
138 +             print("Täyttöaste: Ei määritetty (tilavuus 0)")
139 +         else:
140 +             print(f"Täyttöaste: {(varasto.saldo / varasto.tilavuus * 100):.1f}%")
```

Commit suggestion ▾ | Add suggestion to batch

Extreme programmingin katselmoinnin menetelmiä

Extreme programmingin katselmoinnin menetelmiä

- Toisin kuin Scrum, eXtreme Programming eli XP määrittelee useita sovelluskehityksen käytänteitä
  - Suuri osa näistä on hyvin tunnettuja parhaita käytänteitä, vietynä äärimmäiseen (extreme) muotoon

Extreme programmingin katselmoinnin menetelmiä

- Toisin kuin Scrum, eXtreme Programming eli XP määrittelee useita sovelluskehityksen käytänteitä
  - Suuri osa näistä on hyvin tunnettuja parhaita käytänteitä, vietynä äärimmäiseen (extreme) muotoon
- Osa käytänteistä tähtää laadun maksimoimiseen, kolmen voidaan ajatella olevan katselmoinnin sukulaisia

Pariohjelmointi

- *Pariohjelmoinnissa* (pair programming) kaksi ohjelmoijaa työskentelee yhdessä samalla koneella

Pariohjelmointi

- *Pariohjelmoinnissa* (pair programming) kaksi ohjelmoijaa työskentelee yhdessä samalla koneella
- Koodia kirjoittava osapuoli toimii *ohjaajana* (driver) ja toinen *navigoijana* (navigator)
  - Roolia vaihdetaan sopivin väliajoin

Pariohjelmointi

- *Pariohjelmoinnissa* (pair programming) kaksi ohjelmoijaa työskentelee yhdessä samalla koneella
- Koodia kirjoittava osapuoli toimii *ohjaajana* (driver) ja toinen *navigoijana* (navigator)
  - Roolia vaihdetaan sopivin väliajoin
- Navigoija tekee koodiin **jatkuvaa katselmointia**

Pariohjelmoinnin etuja

- Parantaa ohjelmoijien kuria ja työhön keskittymistä

Pariohjelmoinnin etuja

- Parantaa ohjelmoijien kuria ja työhön keskittymistä
- Hyvä oppimisen väline
  - ohjelmoijat oppivat toisiltaan erityisesti noviisit kokeneimmilta
  - järjestelmän tietyn osan tuntee aina useampi ohjelmoija

Pariohjelmoinnin etuja

- Parantaa ohjelmoijien kuria ja työhön keskittymistä
- Hyvä oppimisen väline
  - ohjelmoijat oppivat toisiltaan erityisesti noviisit kokeneimmilta
  - järjestelmän tietyn osan tuntee aina useampi ohjelmoija
- Tutkimusten mukaan parantaa laatua jossain määrin, mutta kokonaistyömäärä kasvaa
  - eniten hyötyä monimutkaisissa tehtävissä ja kun tavoitteena myös oppiminen

Pariohjelmoinnin etuja

- Parantaa ohjelmoijien kuria ja työhön keskittymistä
- Hyvä oppimisen väline
  - ohjelmoijat oppivat toisiltaan erityisesti noviisit kokeneimmilta
  - järjestelmän tietyn osan tuntee aina useampi ohjelmoija
- Tutkimusten mukaan parantaa laatua jossain määrin, mutta kokonaistyömäärä kasvaa
  - eniten hyötyä monimutkaisissa tehtävissä ja kun tavoitteena myös oppiminen
- Myönteinen vaikutus tiimidynamiikkaan
  - kytkeytyy psykologiseen turvallisuuteen: edellyttää sitä ja toisaalta vahvistaa sitä
  - jatkuva pariohjelmointi koetaan kuormittavaksi, siksi usein valikoidusti

Pariohjelmoinnin yleisyys

- Pariohjelmointi tai useamman ihmisen versio siitä, *mob-programming* on melko yleistä

Pariohjelmoinnin yleisyys

- Pariohjelmointi tai useamman ihmisen versio siitä, *mob-programming* on melko yleistä
- Määritelmän mukaista systemaattista pariohjelmointia tehdään aika harvassa paikassa aamusta iltaan

Pariohjelmoinnin yleisyys

- Pariohjelmointi tai useamman ihmisen versio siitä, *mob-programming* on melko yleistä
- Määritelmän mukaista systemaattista pariohjelmointia tehdään aika harvassa paikassa aamusta iltaan
- Yleensä ohjelmoidaan yksin, mutta spontaania pariutumista ja ryhmäytymistä tapahtuu
  - erityisesti teknisesti haasteellisissa koodin osissa
  - tai jos kyse itselle tuntemattomasta osasta koodia

Yhteisomistajuus

- *Koodin yhteisomistajuus* (collective code ownership): kukaan yksittäinen ohjelmoija ei hallitse yksin mitään kohtaa koodista
  - Kaikilla lupa tehdä muutoksia mihin tahansa kohtaan koodia

Yhteisomistajuus

- *Koodin yhteisomistajuus* (collective code ownership): kukaan yksittäinen ohjelmoija ei hallitse yksin mitään kohtaa koodista
  - Kaikilla lupa tehdä muutoksia mihin tahansa kohtaan koodia
- Pariohjelmointi tukee yhteisomistajuutta

Yhteisomistajuus

- *Koodin yhteisomistajuus* (collective code ownership): kukaan yksittäinen ohjelmoija ei hallitse yksin mitään kohtaa koodista
  - Kaikilla lupa tehdä muutoksia mihin tahansa kohtaan koodia
- Pariohjelmointi tukee yhteisomistajuutta
- Yhteisomistajuudessa on riskinsä: koodia tuntematon voi saada pahaa jälkeä aikaan
  - XP eliminoi riskejä testauksiin liittyvillä käytänteillä
  - kun virhe pääsee läpi, selvitetään onko testeissä samanlainen aukko muuallakin

Yhteisomistajuus

- *Koodin yhteisomistajuus* (collective code ownership): kukaan yksittäinen ohjelmoija ei hallitse yksin mitään kohtaa koodista
  - Kaikilla lupa tehdä muutoksia mihin tahansa kohtaan koodia
- Pariohjelmointi tukee yhteisomistajuutta
- Yhteisomistajuudessa on riskinsä: koodia tuntematon voi saada pahaa jälkeä aikaan
  - XP eliminoi riskejä testauksiin liittyvillä käytänteillä
  - kun virhe pääsee läpi, selvitetään onko testeissä samanlainen aukko muuallakin
- *Ohjelmointistandardi* (coding standards): tiimi määrittelee koodityylin, johon kaikki ohjelmoijat sitoutuvat
  - Tyylillä tarkoitetaan nimeämiskäytäntöä, koodin muotoilua ja myös tiettyjä ohjelman rakenteeseen liittyviä seikkoja

Yhteisomistajuus

- *Koodin yhteisomistajuus* (collective code ownership): kukaan yksittäinen ohjelmoija ei hallitse yksin mitään kohtaa koodista
  - Kaikilla lupa tehdä muutoksia mihin tahansa kohtaan koodia
- Pariohjelmointi tukee yhteisomistajuutta
- Yhteisomistajuudessa on riskinsä: koodia tuntematon voi saada pahaa jälkeä aikaan
  - XP eliminoi riskejä testauksiin liittyvillä käytänteillä
  - kun virhe pääsee läpi, selvitetään onko testeissä samanlainen aukko muuallakin
- *Ohjelmointistandardi* (coding standards): tiimi määrittelee koodityylin, johon kaikki ohjelmoijat sitoutuvat
  - Tyylillä tarkoitetaan nimeämiskäytäntöä, koodin muotoilua ja myös tiettyjä ohjelman rakenteeseen liittyviä seikkoja
- Noudattamista kontrolloidaan katselmoimalla sekä automaattisesti staattisen analyysin työkaluilla kuten Pylintillä

TAUKO 10 min

Testaus

Testaus

- Ohjelmiston osoittaminen virheettömäksi on mahdotonta

Testaus

- Ohjelmiston osoittaminen virheettömäksi on mahdotonta
- Testauksen tarkoituksena vakuuttaa asiakas ja kehittäjät siitä, että ohjelmisto on *tarpeeksi hyvä* käytettäväksi

Testaus

- Ohjelmiston osoittaminen virheettömäksi on mahdotonta
- Testauksen tarkoituksena vakuuttaa asiakas ja kehittäjät siitä, että ohjelmisto on *tarpeeksi hyvä* käytettäväksi
- Testauksella on kaksi eriävää tavoitetta
  - osoittaa, että ohjelmisto täyttää sille asetetut vaatimukset
  - löytää ohjelmistosta virheitä

Testaus

- Ohjelmiston osoittaminen virheettömäksi on mahdotonta
- Testauksen tarkoituksena vakuuttaa asiakas ja kehittäjät siitä, että ohjelmisto on *tarpeeksi hyvä* käytettäväksi
- Testauksella on kaksi eriävää tavoitetta
  - osoittaa, että ohjelmisto täyttää sille asetetut vaatimukset
  - löytää ohjelmistosta virheitä
- Tähtää ohjelman *ulkoisen laadun* (external quality) eli käyttäjän kokeman laadun parantamiseen
  - toteuttaako halutun toiminnallisuuden
  - onko riittävän bugiton käytettäväksi
  - sopiiko sovellus sen käyttötarkoitukseen

Testauksen tasot

Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta

Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus

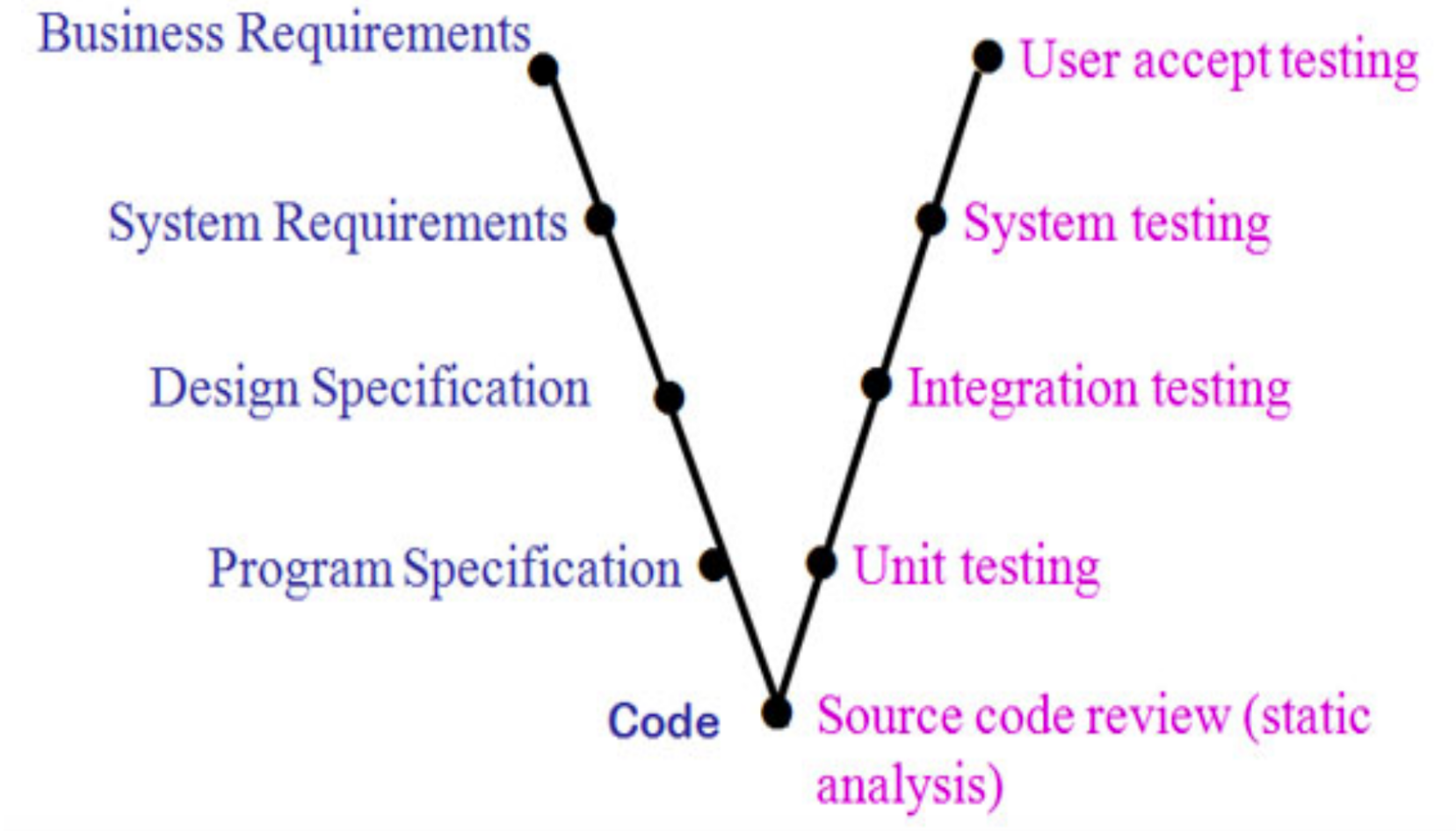
Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin

Testauksen tasot

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin
- *Käyttäjän hyväksymistestaus* (user acceptance testing)
  - Loppukäyttäjän tuotteelle suorittama testaus

“V-malli”


Business Requirements
System Requirements
Design Specification
Program Specification
Code
User accept testing
System testing
Integration testing
Unit testing
Source code review (static analysis)

Järjestelmätestaus

- Tarkoitus varmistaa, että järjestelmä toimii vaatimuksiin kirjatulla tavalla
  - Perinteisesti erilliset testaajat tai laadunhallintatiimi
  - Ketterässä ihanteena, että testaus kuuluu koko (cross functional) tiimille

Järjestelmätestaus

- Tarkoitus varmistaa, että järjestelmä toimii vaatimuksiin kirjatulla tavalla
  - Perinteisesti erilliset testaajat tai laadunhallintatiimi
  - Ketterässä ihanteena, että testaus kuuluu koko (cross functional) tiimille
- Testaus tapahtuu yleensä ilman tietoa järjestelmän sisäisestä rakenteesta eli kyseessä *black box* -testaus

Järjestelmätestaus

- Tarkoitus varmistaa, että järjestelmä toimii vaatimuksiin kirjatulla tavalla
  - Perinteisesti erilliset testaajat tai laadunhallintatiimi
  - Ketterässä ihanteena, että testaus kuuluu koko (cross functional) tiimille
- Testaus tapahtuu yleensä ilman tietoa järjestelmän sisäisestä rakenteesta eli kyseessä *black box* -testaus
- Testataan järjestelmää saman rajapinnan kautta, jonka kautta järjestelmää käytetään

Järjestelmätestaus

- Tarkoitus varmistaa, että järjestelmä toimii vaatimuksiin kirjatulla tavalla
  - Perinteisesti erilliset testaajat tai laadunhallintatiimi
  - Ketterässä ihanteena, että testaus kuuluu koko (cross functional) tiimille
- Testaus tapahtuu yleensä ilman tietoa järjestelmän sisäisestä rakenteesta eli kyseessä *black box* -testaus
- Testataan järjestelmää saman rajapinnan kautta, jonka kautta järjestelmää käytetään
- Tarkastelevat sovelluksen toiminnalisuutta kaikilla tasoilla käyttöliittymästä sovelluslogiikkaan ja tietokantaan
  - Käytetään nimitystä *end to end -testaus*

Järjestelmätestaus

- Perustuu järjestelmän potentiaalisiin käyttöskenaarioihin
  - user storyinä olevista vaatimuksista helppo muotoilla testejä
  - jos storyillä hyväksymiskriteeriot, tilanne on vieläkin parempi

Järjestelmätestaus

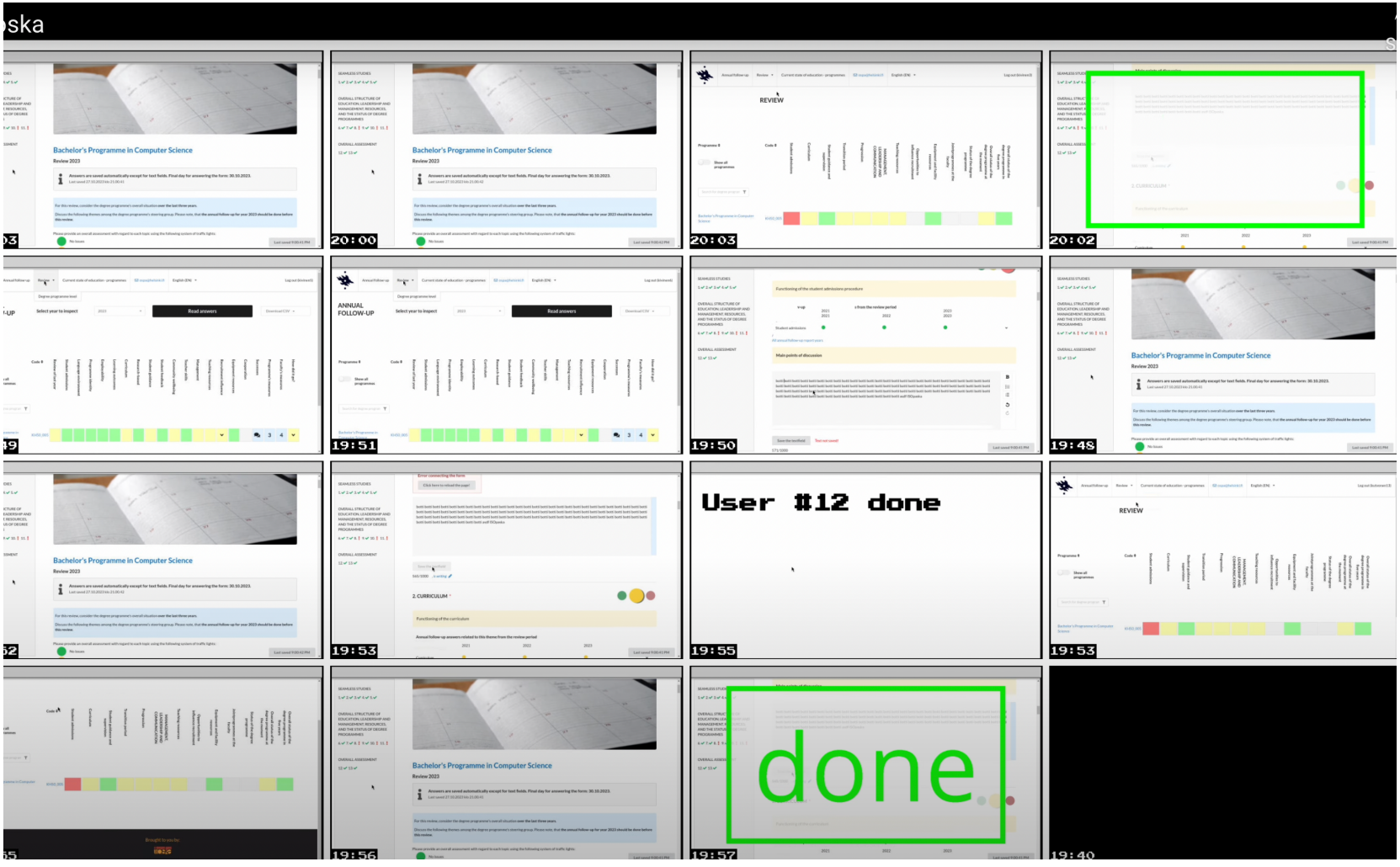
- Perustuu järjestelmän potentiaalisiin käyttöskenaarioihin
  - user storyinä olevista vaatimuksista helppo muotoilla testejä
  - jos storyillä hyväksymiskriteeriot, tilanne on vieläkin parempi
- Kutsutaan myös *toiminnallisiksi testeiksi* (functional test)

Järjestelmätestaus

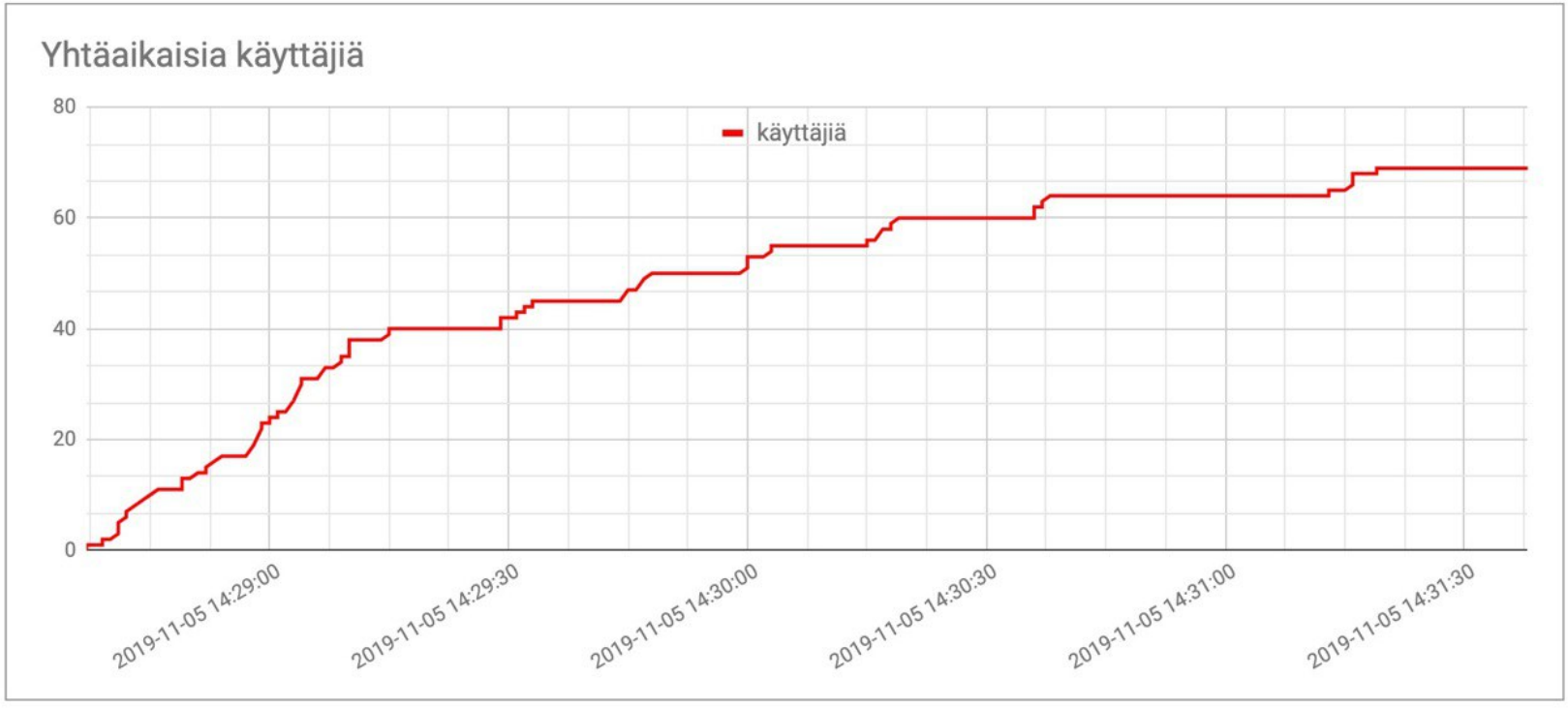
- Perustuu järjestelmän potentiaalisiin käyttöskenaarioihin
  - user storyinä olevista vaatimuksista helppo muotoilla testejä
  - jos storyillä hyväksymiskriteeriot, tilanne on vieläkin parempi
- Kutsutaan myös *toiminnallisiksi testeiksi* (functional test)
- Toiminnallisen testauksen lisäksi järjestelmätestaukseen kuuluu mm.
  - Käytettävyystestaus
  - Suorituskykytestaus
  - Kuormitustestaus
  - Tietoturvan testaus
  - Saavutettavuuden testaus

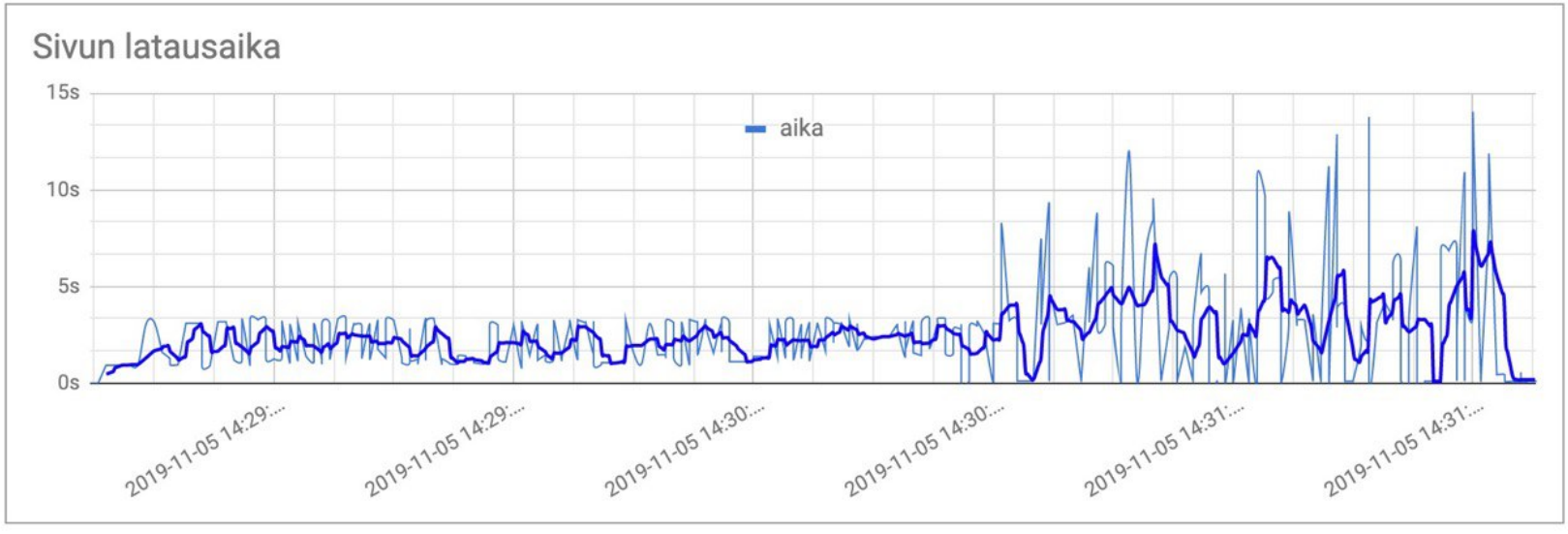
Kuormitustestaus: esimerkki

Kuormitustestaus: esimerkki

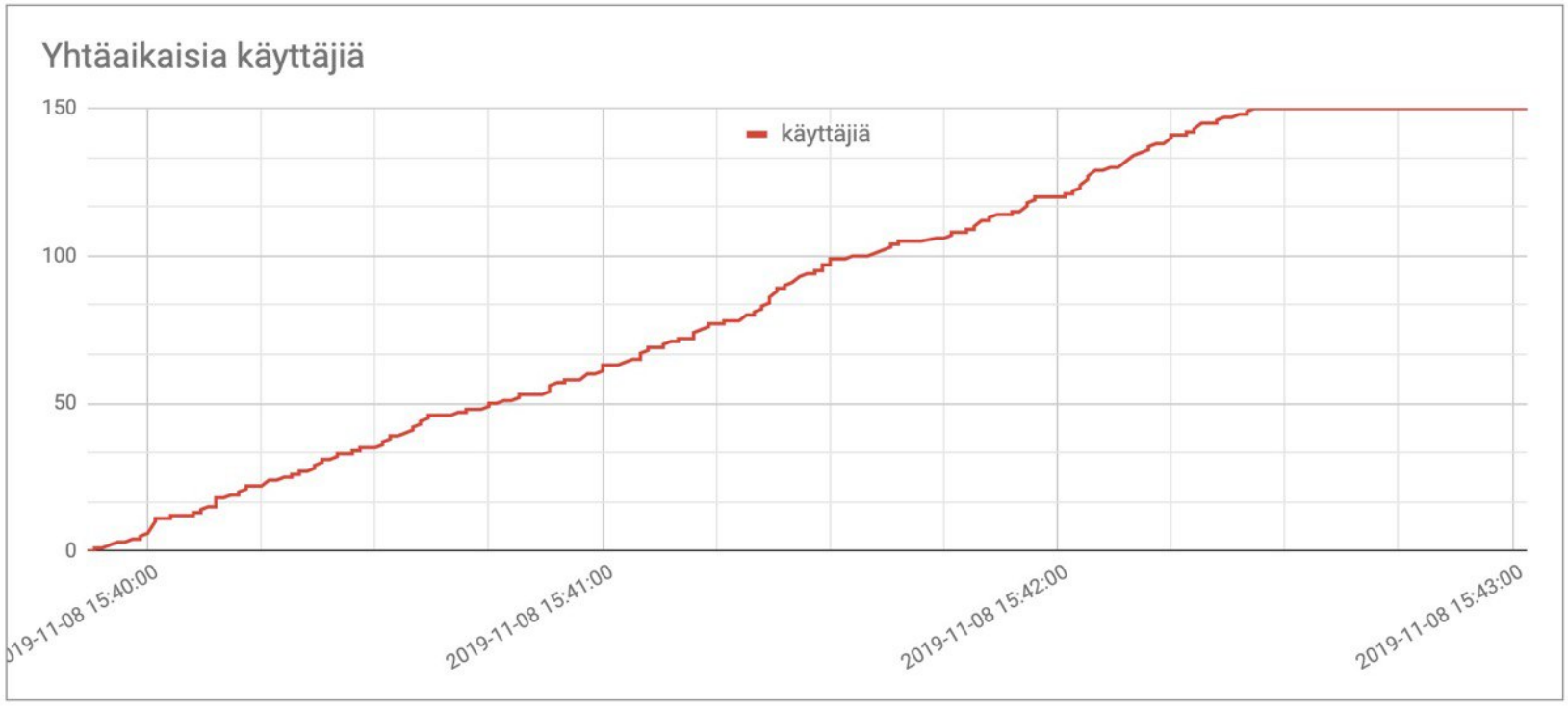


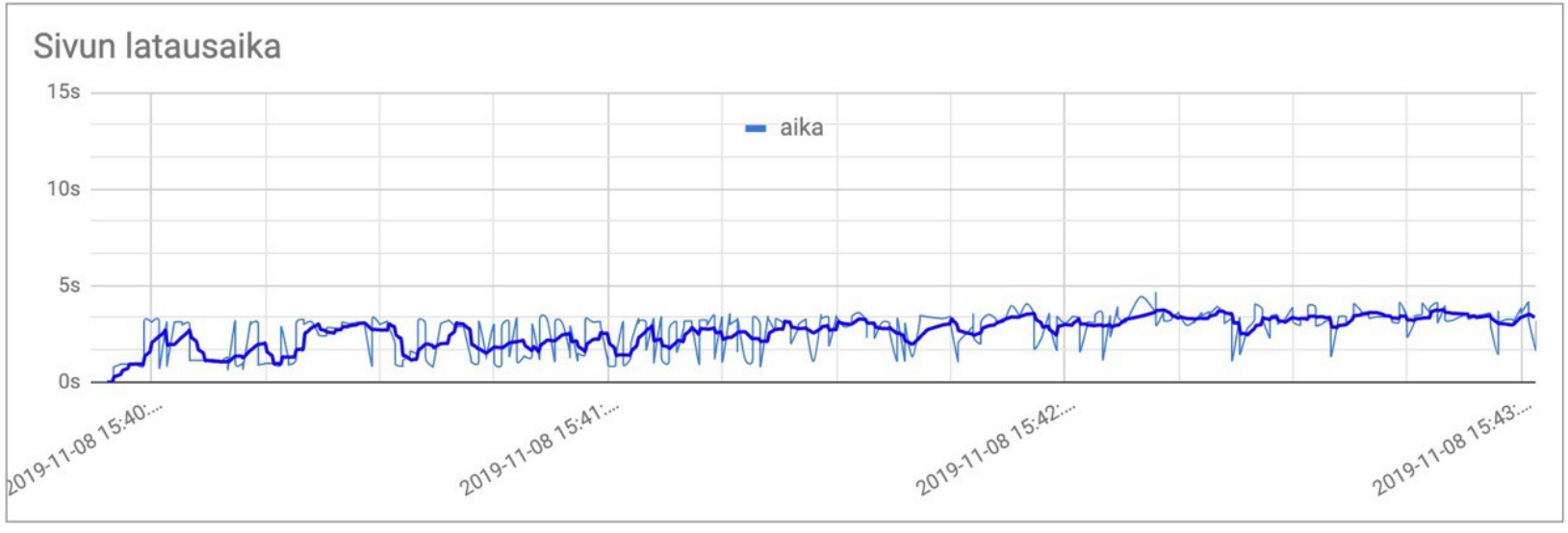
Kuormitustestaus: ennen





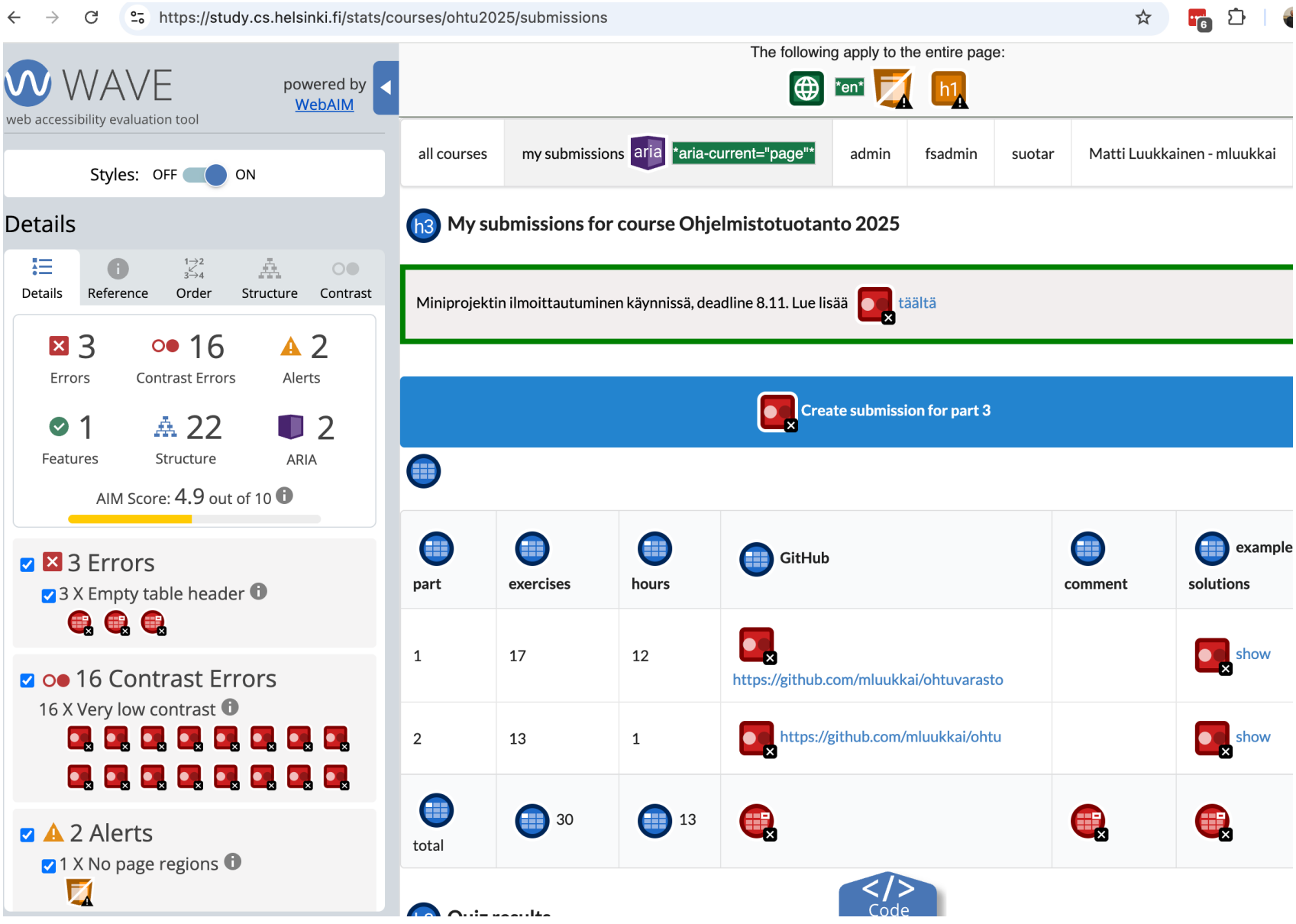
Kuormitustestaus: jälkeen





Saavutettavuuden testaaminen: palautusjärjestelmä

Saavutettavuuden testaaminen: palautusjärjestelmä



Testitapausten valinta

- Yksi *Testitapaus* testaa toiminnallisuutta joillakin syötteillä

Testitapausten valinta

- Yksi *Testitapaus* testaa toiminnallisuutta joillakin syötteillä
- Kattava testaaminen on mahdotonta ja testaus työlästä
  - Oleellista löytää kohtuullisen kokoinen *testitapausten joukko*, jonka avulla löytyy suuri määrä virheitä

Testitapausten valinta

- Yksi *Testitapaus* testaa toiminnallisuutta joillakin syötteillä
- Kattava testaaminen on mahdotonta ja testaus työlästä
  - Oleellista löytää kohtuullisen kokoinen *testitapausten joukko*, jonka avulla löytyy suuri määrä virheitä
- Useat syötteet ohjelmiston toiminnan kannalta samanlaisia
  - nämä muodostavat *ekvivalenssiluokan*

Testitapausten valinta

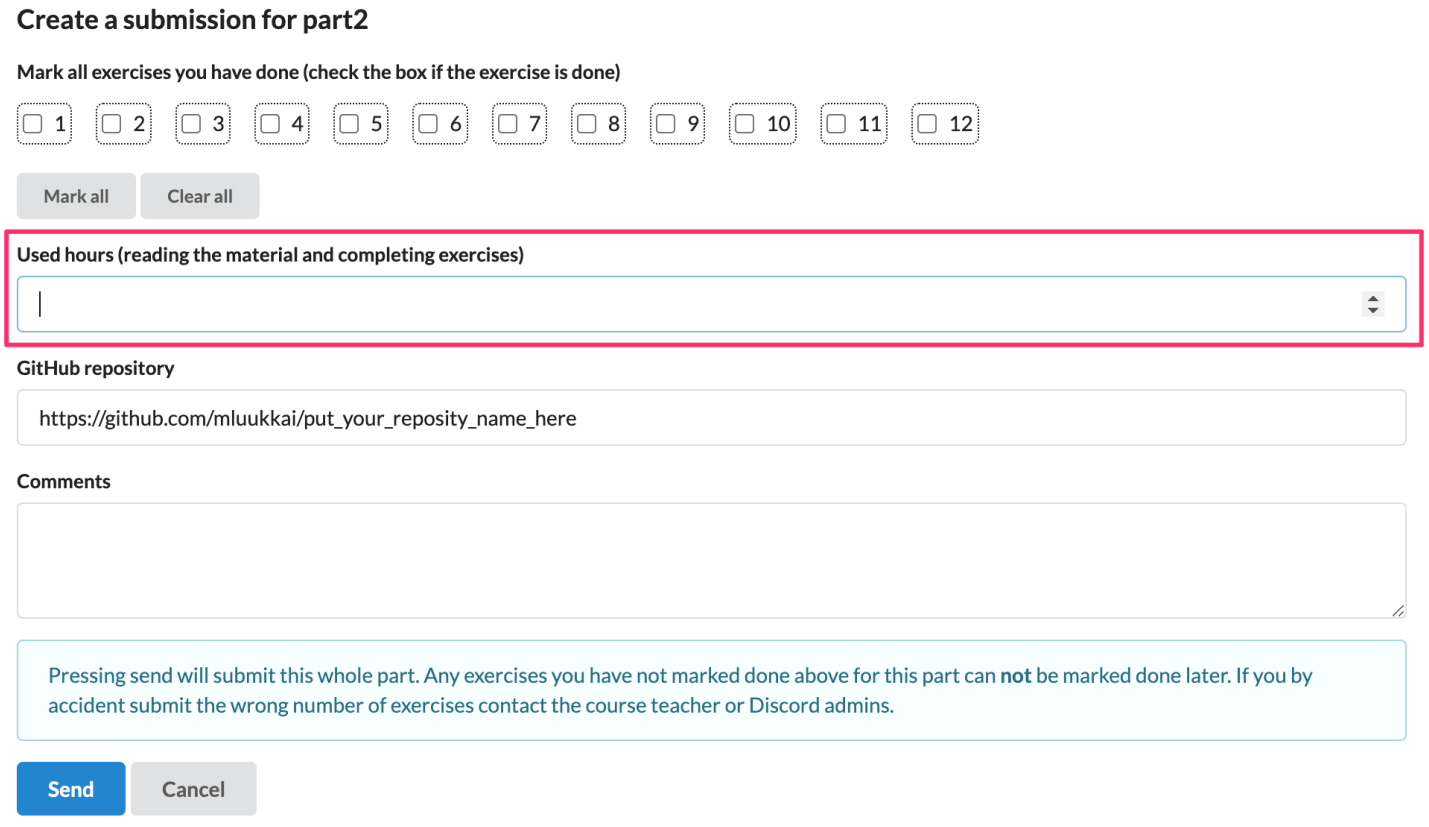
- Yksi *Testitapaus* testaa toiminnallisuutta joillakin syötteillä
- Kattava testaaminen on mahdotonta ja testaus työlästä
  - Oleellista löytää kohtuullisen kokoinen *testitapausten joukko*, jonka avulla löytyy suuri määrä virheitä
- Useat syötteet ohjelmiston toiminnan kannalta samanlaisia
  - nämä muodostavat *ekvivalenssiluokan*
- Jaetaan syötteet ekvivalenssiluokkiin ja tehdään yksi testitapaus jokaiseta ekvivalenssiluokkasta

Testitapausten valinta

- Yksi *Testitapaus* testaa toiminnallisuutta joillakin syötteillä
- Kattava testaaminen on mahdotonta ja testaus työlästä
  - Oleellista löytää kohtuullisen kokoinen *testitapausten joukko*, jonka avulla löytyy suuri määrä virheitä
- Useat syötteet ohjelmiston toiminnan kannalta samanlaisia
  - nämä muodostavat *ekvivalenssiluokan*
- Jaetaan syötteet ekvivalenssiluokkiin ja tehdään yksi testitapaus jokaiseta ekvivalenssiluokkasta
- Erityisen kiinnostavia syötearvoja ovat ekvivalenssiluokkien väliset *raja-arvot*

Testisyötteiden valinta: palautussovellus

- Mitä testitapauksia kannattaisi valita palautussovelluksen testaamiseen?



Testisyötteiden valinta: palautussovellus

- Tunnit
  - tyhjä
  - negatiivinen
  - nolla
  - positiivinen järkevä arvo (esim. 5)
  - positiivinen mutta epärealistinen (esim. 1000)
  - merkkejä sisältävä syöte

Testisyötteiden valinta: palautussovellus

- Tunnit
  - tyhjä
  - negatiivinen
  - nolla
  - positiivinen järkevä arvo (esim. 5)
  - positiivinen mutta epärealistinen (esim. 1000)
  - merkkejä sisältävä syöte
- Repositorio
  - tyhjä
  - validi repositoriolinkki
  - epävalidi merkkijono

Yksikkötestaus

Yksikkötestaus

- Kohteena siis yksittäiset metodit ja luokat
  - *Developer testing*: sovelluskehittäjien vastuulla

Yksikkötestaus

- Kohteena siis yksittäiset metodit ja luokat
  - *Developer testing*: sovelluskehittäjien vastuulla
- Testattavan koodin rakenne otetaan huomioon testejä laatiessa, *lasilaatikkotestausta* (white box testing)

Yksikkötestaus

- Kohteena siis yksittäiset metodit ja luokat
  - *Developer testing*: sovelluskehittäjien vastuulla
- Testattavan koodin rakenne otetaan huomioon testejä laatiessa, *lasilaatikkotestausta* (white box testing)
- Päätarkoitus *sisäisen laadun* (internal quality) kontrollointi
  - onko virheiden jäljitys ja korjaaminen helppoa
  - onko koodia helppo laajentaa ja jatkokehittää
  - pystytäänkö koodin toiminnallisuuden oikeellisuus varmistamaan muutoksia tehtäessä

Yksikkötestaus

- Iteratiivisessa kehityksessä sisäisellä laadulla suuri merkitys
  - ohjelmistoa laajennetaan koko ajan

Yksikkötestaus

- Iteratiivisessa kehityksessä sisäisellä laadulla suuri merkitys
  - ohjelmistoa laajennetaan koko ajan
- Edesauttaa myös ulkoista eli asiakkaan kokemaa laatua
  - Yksikkötestit voivat eliminoida joitain asiakkaalle näkyviä virheitä, joita järjestelmätestauksen testitapaukset eivät löydä

Yksikkötestaus

- Iteratiivisessa kehityksessä sisäisellä laadulla suuri merkitys
  - ohjelmistoa laajennetaan koko ajan
- Edesauttaa myös ulkoista eli asiakkaan kokemaa laatua
  - Yksikkötestit voivat eliminoida joitain asiakkaalle näkyviä virheitä, joita järjestelmätestauksen testitapaukset eivät löydä
- Bugit parempi paikallistaa ajoissa
  - yksikkötestauksessa löydetty bugi on yleensä helpompi ja nopeampi korjata kuin järjestelmä- tai integraatiotestauksessa löytyvä

Yksikkötestaus

- Iteratiivisessa kehityksessä sisäisellä laadulla suuri merkitys
  - ohjelmistoa laajennetaan koko ajan
- Edesauttaa myös ulkoista eli asiakkaan kokemaa laatua
  - Yksikkötestit voivat eliminoida joitain asiakkaalle näkyviä virheitä, joita järjestelmätestauksen testitapaukset eivät löydä
- Bugit parempi paikallistaa ajoissa
  - yksikkötestauksessa löydetty bugi on yleensä helpompi ja nopeampi korjata kuin järjestelmä- tai integraatiotestauksessa löytyvä
- Koska yksikkötestejä joudutaan suorittamaan moneen kertaan, tulee niiden suorittaminen *automatisoida*

Mitä tulisi (yksikkö)testata?

- Automatisoidun testauksen esi-isän Kent Beckin vastaus:
  - “Do I have to write a test for everything?”
  - “No, just test everything that could reasonably break”

Mitä tulisi (yksikkö)testata?

- Automatisoidun testauksen esi-isän Kent Beckin vastaus:
  - “Do I have to write a test for everything?”
  - “No, just test everything that could reasonably break”
- Optimitilanteessa testitapaukset
  - kaikkien metodit ja niiden kutsukombinaatiot hyväksyttävillä parametrien arvoilla
  - ja virheellisillä parametrien arvoilla

Mitä tulisi (yksikkö)testata?

- Automatisoidun testauksen esi-isän Kent Beckin vastaus:
  - “Do I have to write a test for everything?”
  - “No, just test everything that could reasonably break”
- Optimitilanteessa testitapaukset
  - kaikkien metodit ja niiden kutsukombinaatiot hyväksyttävillä parametrien arvoilla
  - ja virheellisillä parametrien arvoilla
- Ekvivalenssiluokat ja niiden raja-arvot kannattaa huomioida
  - testien parametrien ekvivalenssiluokat ja raja-arvot päätelтävissä koodista

Ominaisuuspohjainen testaus

- *Property-based testing*: syötteitä ei valita käsin, vaan kuvataan ominaisuus, jonka pitää päteä kaikilla syötteillä
  - kirjasto generoi satoja syötteitä ja etsii ominaisuuden rikkovia

```
@given(st.integers(min_value=0, max_value=100),
       st.integers(min_value=-10, max_value=200))
def test_ottaminen_ei_hukkaa_tavaraa(alku_saldo, maara):
    varasto = Varasto(100, alku_saldo)
    otettu = varasto.ota_varastosta(maara)

    assert varasto.saldo >= 0
    assert otettu + varasto.saldo == alku_saldo
```

- Pythonissa Hypothesis-kirjasto, joka myös pienentää virheen aiheuttavan syötteen yksinkertaisimmaksi

Testauskattavuus

- Testien laajuutta voidaan mitata *testauskattavuuden* (test coverage) käsitteellä

Testauskattavuus

- Testien laajuutta voidaan mitata *testauskattavuuden* (test coverage) käsitteellä
- Muutamaa eri tyyppiä
  - **rivikattavuus**
  - **haarautumakattavuus**
  - ehtokattavuus
  - polkukattavuus

Testauskattavuus

- Testien laajuutta voidaan mitata *testauskattavuuden* (test coverage) käsitteellä
- Muutamaa eri tyyppiä
  - **rivikattavuus**
  - **haarautumakattavuus**
  - ehtokattavuus
  - polkukattavuus
- Rivi- ja haarautumakattavuudelle hyvä työkalutuki, esim. coverage Pythonille

Coverage for **src/varasto.py**: 79%

33 statements | 27 run | 6 missing | 0 excluded | 4 partial

« prev ^ index » next coverage.py v7.10.7, created at 2025-11-01 17:23 +0200

```
1  class Varasto:
2      def __init__(self, tilavuus, alku_saldo = 0):
3          self.tilavuus = self._tilavuus_alussa(tilavuus)
4          self.saldo = self._saldo_alussa(alku_saldo)
5
6      def paljonko_mahtuu(self):
7          return self.tilavuus - self.saldo
8
9      def lisaa_varastoon(self, maara):
10         if maara < 0:
11             return
12         if maara <= self.paljonko_mahtuu():
13             self.saldo = self.saldo + maara
14         else:
15             self.saldo = self.tilavuus
16
17     def ota_varastosta(self, maara):
18         if maara < 0:
19             return 0.0
20         if maara > self.saldo:
21             kaikki_mita_voidaan = self.saldo
22             self.saldo = 0.0
23
24             return kaikki_mita_voidaan
25
26         self.saldo = self.saldo - maara
27
28         return maara
```

Mitä kattavuus kertoo?

- Kattavuus kertoo, mitä **ei** ole testattu
  - korkea kattavuus ei vielä kerro, että testit tarkistavat jotain järkevää
  - kattavuuden ja testien virheiden löytämiskyvyn välinen yhteys on tutkimuksissa heikko

Mitä kattavuus kertoo?

- Kattavuus kertoo, mitä **ei** ole testattu
  - korkea kattavuus ei vielä kerro, että testit tarkistavat jotain järkevää
  - kattavuuden ja testien virheiden löytämiskyvyn välinen yhteys on tutkimuksissa heikko
- Kattavuutta ei kannata asettaa tavoitteeksi (Goodhartin laki)
  - Googlen ohjeistus: 60 % hyväksyttävä, 75 % suositeltava, 90 % esimerkillinen

Mitä kattavuus kertoo?

- Kattavuus kertoo, mitä **ei** ole testattu
  - korkea kattavuus ei vielä kerro, että testit tarkistavat jotain järkevää
  - kattavuuden ja testien virheiden löytämiskyvyn välinen yhteys on tutkimuksissa heikko
- Kattavuutta ei kannata asettaa tavoitteeksi (Goodhartin laki)
  - Googlen ohjeistus: 60 % hyväksyttävä, 75 % suositeltava, 90 % esimerkillinen
- Testien hyvyyttä voi arvioida *mutaatiotestauksella*
  - koodiin istutetaan pieniä bugeja: huomaavatko testit ne?
  - työkaluja: mutmut (Python), PIT (Java), Stryker (JavaScript)
  - hyödyllinen erityisesti tekoälyagentin kirjoittamien testien arvioinnissa

Ohjelmiston integraatio

Ohjelmiston integraatio

- Järjestelmän yksittäiset, erillään yksikkötestatut luokat tulee integroida toimivaksi kokonaisuudeksi

Ohjelmiston integraatio

- Järjestelmän yksittäiset, erillään yksikkötestatut luokat tulee integroida toimivaksi kokonaisuudeksi
- Integroinnin yhteydessä suoritetaan integrointitestaus
  - Toimivatko komponentit yhdessä?

Ohjelmiston integraatio

- Järjestelmän yksittäiset, erillään yksikkötestatut luokat tulee integroida toimivaksi kokonaisuudeksi
- Integroinnin yhteydessä suoritetaan integrointitestaus
  - Toimivatko komponentit yhdessä?
- Kaksi lähestymistapaa:
  - rakenteeseen perustuva
  - toiminnallisuuksiin perustuva

Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan

Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan

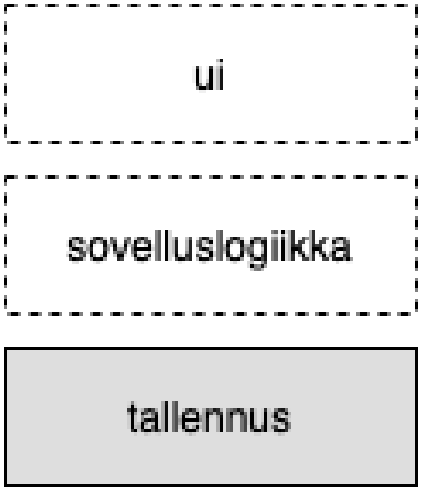
Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan



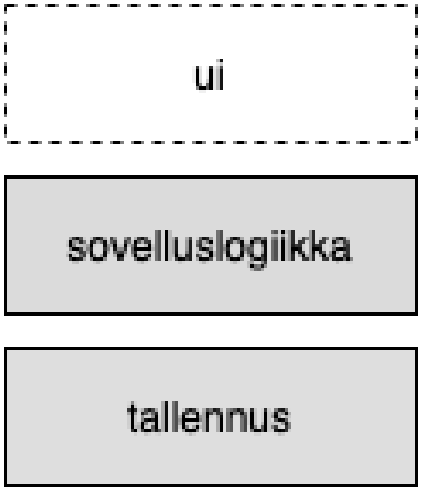
Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan



Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan



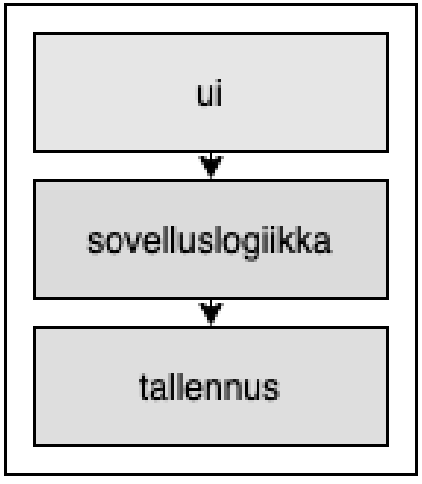
Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan



Rakenteeseen perustuva integraatio

- Integroidaan arkkitehtuurikomponentti kerrallaan
- Verkkokaupassa: sovelluslogiikan luokat, käyttöliittymän luokat ja tietokantarajapinta ensin omina kokonaisuuksinaan



Ohjelmiston integraatio

- Rakenteeseen perustuva integraatio on vesiputuoksen toimintatapa

Ohjelmiston integraatio

- Rakenteeseen perustuva integraatio on vesiputuoksen toimintatapa
- Tämän tyylinen *big bang* -integraatio on osoittautunut todella riskialttiiksi
  - Seurauksena usein ns. **integraatiohelvetti**

Ohjelmiston integraatio

- Rakenteeseen perustuva integraatio on vesiputuoksen toimintatapa
- Tämän tyylinen *big bang* -integraatio on osoittautunut todella riskialttiiksi
  - Seurauksena usein ns. **integraatiohelvetti**
- Moderni ohjelmistotuotanto suosii ns. *jatkuvaa integraatiota*
  - Hyvin tiheässä tahdissa tapahtuvaa ominaisuuksiin perustuvaa integrointia

Ominaisuuksiin perustuva integraatio

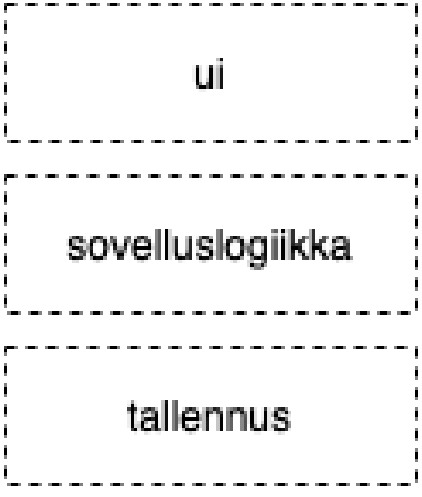
- Toiminnallisuus kerrallaan

Ominaisuuksiin perustuva integraatio

- Toiminnallisuus kerrallaan
- Esim. integroidaan kerrallaan kaikki storyn *lisää tuote ostoskoriin* toteuttava koodi
  - Tämän jälkeen integroidaan seuraavan storyn koodi

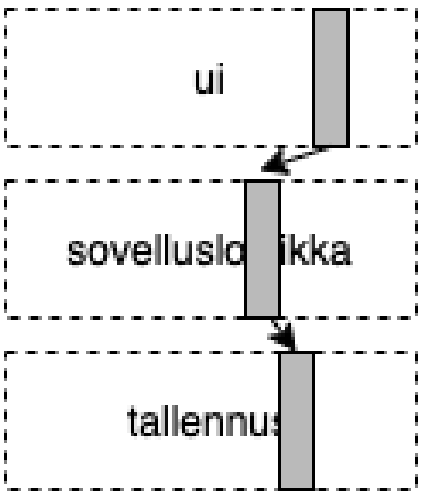
Ominaisuuksiin perustuva integraatio

- Toiminnallisuus kerrallaan
- Esim. integroidaan kerrallaan kaikki storyn *lisää tuote ostoskoriin* toteuttava koodi
  - Tämän jälkeen integroidaan seuraavan storyn koodi



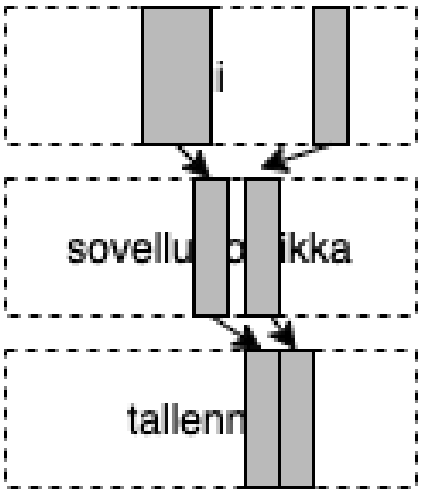
Ominaisuuksiin perustuva integraatio

- Toiminnallisuus kerrallaan
- Esim. integroidaan kerrallaan kaikki storyn *lisää tuote ostoskoriin* toteuttava koodi
  - Tämän jälkeen integroidaan seuraavan storyn koodi



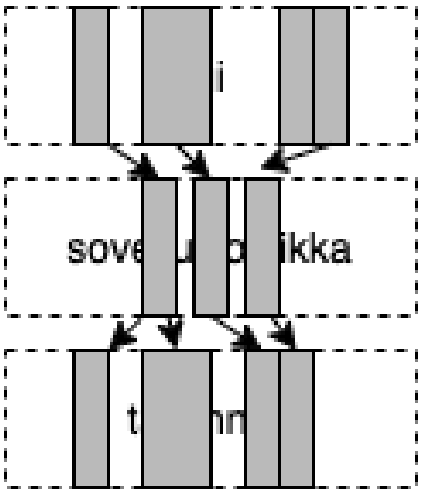
Ominaisuuksiin perustuva integraatio

- Toiminnallisuus kerrallaan
- Esim. integroidaan kerrallaan kaikki storyn *lisää tuote ostoskoriin* toteuttava koodi
  - Tämän jälkeen integroidaan seuraavan storyn koodi



Ominaisuuksiin perustuva integraatio

- Toiminnallisuus kerrallaan
- Esim. integroidaan kerrallaan kaikki storyn *lisää tuote ostoskoriin* toteuttava koodi
  - Tämän jälkeen integroidaan seuraavan storyn koodi



Integraatiotestauksen käytänteitä

- Integraatiotesteissä tarvitaan usein oikea tietokanta tai muu ulkoinen palvelu
  - kevyt korvaava tietokanta (esim. SQLite) voi peittää ongelmia
  - nykyään käynnistetään oikea tietokanta testien ajaksi kontissa (esim. Testcontainers)

Integraatiotestauksen käytänteitä

- Integraatiotesteissä tarvitaan usein oikea tietokanta tai muu ulkoinen palvelu
  - kevyt korvaava tietokanta (esim. SQLite) voi peittää ongelmia
  - nykyään käynnistetään oikea tietokanta testien ajaksi kontissa (esim. Testcontainers)
- Useista erikseen kehitettävistä palveluista koostuva sovellus: *sopimustestaus* (contract testing)
  - palvelun käyttäjä kirjaa odottamansa pyynnöt ja vastaukset
  - palvelun tarjoaja testaa erikseen täyttävänsä sopimuksen
  - esim. Pact

Regressiotestaus

- Iteratiivisessa ohjelmistotuotannossa, jokainen iteraatio tuottaa ohjelmistoon uusia ominaisuuksia
  - Samalla tulee huolehtia, että ei rikota jo toimivia osia

Regressiotestaus

- Iteratiivisessa ohjelmistotuotannossa, jokainen iteraatio tuottaa ohjelmistoon uusia ominaisuuksia
  - Samalla tulee huolehtia, että ei rikota jo toimivia osia
- Testit on suoritettava uudelleen aina kun ohjelmistoon tehdään muutoksia
- Tätä käytäntöä sanotaan *regressiotestaukseksi*

Regressiotestaus

- Iteratiivisessa ohjelmistotuotannossa, jokainen iteraatio tuottaa ohjelmistoon uusia ominaisuuksia
  - Samalla tulee huolehtia, että ei rikota jo toimivia osia
- Testit on suoritettava uudelleen aina kun ohjelmistoon tehdään muutoksia
- Tätä käytäntöä sanotaan *regressiotestaukseksi*
- Regressiotestijoukko koostuu kaikista ohjelmistolle tehdyistä testeistä
  - sisältää yksikkö-, integraatio- ja järjestelmätesteistä

Regressiotestaus

- Iteratiivisessa ohjelmistotuotannossa, jokainen iteraatio tuottaa ohjelmistoon uusia ominaisuuksia
  - Samalla tulee huolehtia, että ei rikota jo toimivia osia
- Testit on suoritettava uudelleen aina kun ohjelmistoon tehdään muutoksia
- Tätä käytäntöä sanotaan *regressiotestaukseksi*
- Regressiotestijoukko koostuu kaikista ohjelmistolle tehdyistä testeistä
  - sisältää yksikkö-, integraatio- ja järjestelmätesteistä
- Testaus on työlästä ja regressiotestauksen tarve tekee siitä entistä työläämpää
  - *Testaus kannattaa automatisoida* mahdollisimman suurissa määrin

Epävakaat testit

- *Flaky test*: menee välillä läpi ja välillä ei, vaikka koodi ei ole muuttunut
  - syynä usein ajoitus, testien suoritusjärjestys, ulkoiset palvelut tai jaettu tila

Epävakaat testit

- *Flaky test*: menee välillä läpi ja välillä ei, vaikka koodi ei ole muuttunut
  - syynä usein ajoitus, testien suoritusjärjestys, ulkoiset palvelut tai jaettu tila
- Jos testit punastuvat satunnaisesti, epäonnistumisia aletaan sivuuttaa
  - silloin myös todelliset virheet jäävät huomaamatta

Epävakaat testit

- *Flaky test*: menee välillä läpi ja välillä ei, vaikka koodi ei ole muuttunut
  - syynä usein ajoitus, testien suoritusjärjestys, ulkoiset palvelut tai jaettu tila
- Jos testit punastuvat satunnaisesti, epäonnistumisia aletaan sivuuttaa
  - silloin myös todelliset virheet jäävät huomaamatta
- Googlella noin 16 % testeistä jossain määrin epävakaita

Epävakaat testit

- *Flaky test*: menee välillä läpi ja välillä ei, vaikka koodi ei ole muuttunut
  - syynä usein ajoitus, testien suoritusjärjestys, ulkoiset palvelut tai jaettu tila
- Jos testit punastuvat satunnaisesti, epäonnistumisia aletaan sivuuttaa
  - silloin myös todelliset virheet jäävät huomaamatta
- Googlella noin 16 % testeistä jossain määrin epävakaita
- Martin Fowler: epävakaa testi heti karanteeniin ja korjataan pikaisesti

Testauksen tasot: kertaus

Testauksen tasot: kertaus

- *Yksikkötestaus* (unit testing)
  - Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- *Integraatiotestaus* (integration testing)
  - Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- *Järjestelmätestaus* (system testing)
  - Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
  - Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
  - Jakautuu useisiin alalajeihin
- *Käyttäjän hyväksymistestaus* (user acceptance testing)
  - Loppukäyttäjän tuotteelle suorittama testaus

Huomenna

Huomenna

- Ketterien menetelmien suosimia testauksen ja laadunhallinnan käytänteitä